10. Props in React

▼ What is Props?

- Props stands for **properties** in React.js.
- Props (short for properties) are a fundamental concept in React.
- Props are used to pass data or information from the **Parent** component to the **Child** Component. (**Unidirectional Flow**)

▼ Why Do We Need Props?

- Reasons why props are important:
  - To pass dynamic data to components
  - To make components reusable with different configurations.
  - To communicate between components (mostly from parent to child)

▼ Key Features of Props

Feature	Description
Read-Only	Props cannot be modified by the receiving component.
Unidirectional	Data flows from **parent to child** (top-down).
Reusable	Makes components customizable and reusable.
Support various types	Strings, numbers, booleans, arrays, objects, etc.
Passed as attributes	Passed like HTML attributes to components.

▼ How to pass props from one component to another

- **Parent Component (Passing Props)**

```
import React from "react";
import Child from "./Child";
```

```
const Parent = () => {
  return (
    <>
      <h1>Parent Component</h1>
      <Child name="John" age={20} />
    </>
  );
};

export default Parent;
```

- **Child Component (Consuming Props)**

```
import React from "react";

const Child = (props) => {
  console.log(props); // {name: 'John', age: 20}
  console.log(typeof props); // Object
  return (
    <>
      <h1>Child Component</h1>
      <h2>Name: {props.name}</h2>
      <h2>Age: {props.age}</h2>
    </>
  );
};

export default Child;
```

▼ What are `props` here?

- It is a parameter that holds an **object of all props** passed to the component.
- You can access individual props using dot notation: `props.name`, `props.age`, etc.

▼ How to pass different data types as props

- You can pass all valid JS values:

```
import React from "react";
import Child from "./Child";

const Parent = () => {
  return (
    <>
      <h1>Parent Component - Passing different types of data as props</
h1>
      <Child
        name="John" //string
        age={20} //number
        isAdmin={true} //boolean
        info={{ city: "London", role: "Dev" }} //object
        skills={["JS", "React"]} //array
        address={null} //null
        status={undefined} //undefined
        symbol={Symbol("id")} //symbol
      />
    </>
  );
};

export default Parent;
```

> **Note**: All non-string values must be enclosed in `{}`. But you can use `{}` for the string also.

▼ Destructuring Props

- Instead of using **props.name**, we can extract props (properties) directly:
- **1st way:**

```
import React from "react";

//! Destructuring props - name, age
const Child = (props) => {
  let { name, age } = props;
  return (
    <>
      <h1>Child Component</h1>
      <h2>Name: {name}</h2>
      <h2>Age: {age}</h2>
    </>
  );
};

export default Child;
```

- **2nd Way:**

```
import React from "react";

//! Destructuring props - name, age
const Child = ({ name, age }) => {
  console.log(name); //John
  console.log(age); //20
  return (
    <>
      <h1>Child Component</h1>
      <h2>Name: {name}</h2>
      <h2>Age: {age}</h2>
    </>
  );
};

export default Child;
```

- This is cleaner and more readable, especially when many props are involved.

▼ Passing Functions as Props

- You can pass the functions as a props to the child component:
- **Passing function from parent component:**

```
import React from "react";
import Child from "./Child";

const Parent = () => {
  //passing function as props
  let sayHello = () => {
    alert("Hello! from the parent");
  };
  return (
    <>
      <h1>Parent Component - Passing functions as a prop to child</h1>
      <Child handleClick={sayHello} />
    </>
  );
};

export default Parent;
```

- **Consuming props - Call the function inside the child component:**

```
import React from "react";

const Child = ({ handleClick }) => {
  return (
    <>
      <h1>Child Component</h1>
      <button onClick={handleClick}>Click me</button>
    </>
  );
};

export default Child;
```

▼ `props.children`

- In React, `props.children` is a special prop that allows a component to receive and render any elements or components that are passed as children within its opening and closing tags.
- `props.children` is a reserved prop name in React.
- It represents the content that is nested between the opening and closing tags of a component.
- This content can be any valid React element, including:
  - Text
  - HTML elements (e.g., `<div>`, `<p>`, `<span>`)
  - Other React components
- **For Example →**
- **Parent Component:**

```
import React from "react";
import ChildComponent from "./ChildComponent";

const ParentComponent = () => {
  return (
    <>
      <h1>Parent Component</h1>
      <ChildComponent>
        <p>This is some text inside the ChildComponent.</p>
        <button>Click Me</button>
      </ChildComponent>
    </>
  );
};

export default ParentComponent;
```

- **Child Component:**

```
import React from "react";

const ChildComponent = (props) => {
  return (
    <>
      <h2>Child Component</h2>
      {props.children}
    </>
  );
};

export default ChildComponent;
```

- **Output:**

▼ Default Props in React

- **Default props** are fallback values that a component uses when **no prop value** is passed by the parent.
- We can pass default props By Using a **Default Parameter in the Function Definition (Recommended Way)**
- This is more of a JavaScript trick, not the official way. It's used when you want to provide a fallback **inline**:
- **For Example →**

```
import React from "react";
```

```
// Set default value directly using parameter destructuring
const User = ({ username = "Guest" }) => {
  return (
    <>
      <h1>User Component</h1>
      <h2>Username: {username}</h2>
    </>
  );
};

export default User;
```

- **Usage:**

```
<User /> // Renders: Username: Guest
<User username="John" /> // Renders: Username: John
```